Detailed Pipeline of LLMSched

From Cluster Observation to Candidate Generation, ILP Refinement, Execution, and Feedback

Research Notes

July 19, 2026

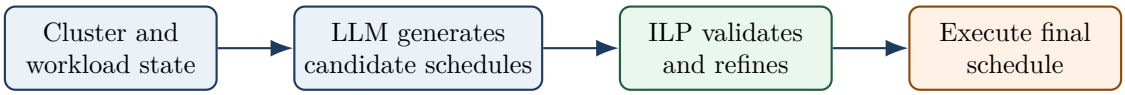
Abstract

This document reconstructs the end-to-end scheduling pipeline described in *LLM-Driven Adaptive Cloud Resource Scheduling: Bridging Reasoning Intelligence With Optimization Guarantees*. The paper combines an LLM-based candidate generator with relevance filtering, constraint-aware prompting, a time-bounded Integer Linear Programming (ILP) refinement stage, strategy caching, fast heuristic adaptation, and closed-loop feedback. The purpose of this report is to make every stage explicit, clarify which part is handled by the LLM and which part is handled by the optimizer, and adapt the design to a simulator in which a scheduler computes the allocation for the next three-minute epoch while the current allocation is executing.

Contents

1 High-Level Interpretation

The main design principle is not that the LLM is itself a complete production scheduler. Instead, the LLM acts as a **strategic candidate generator**; exact feasibility and final assignment are delegated to a conventional optimization stage.



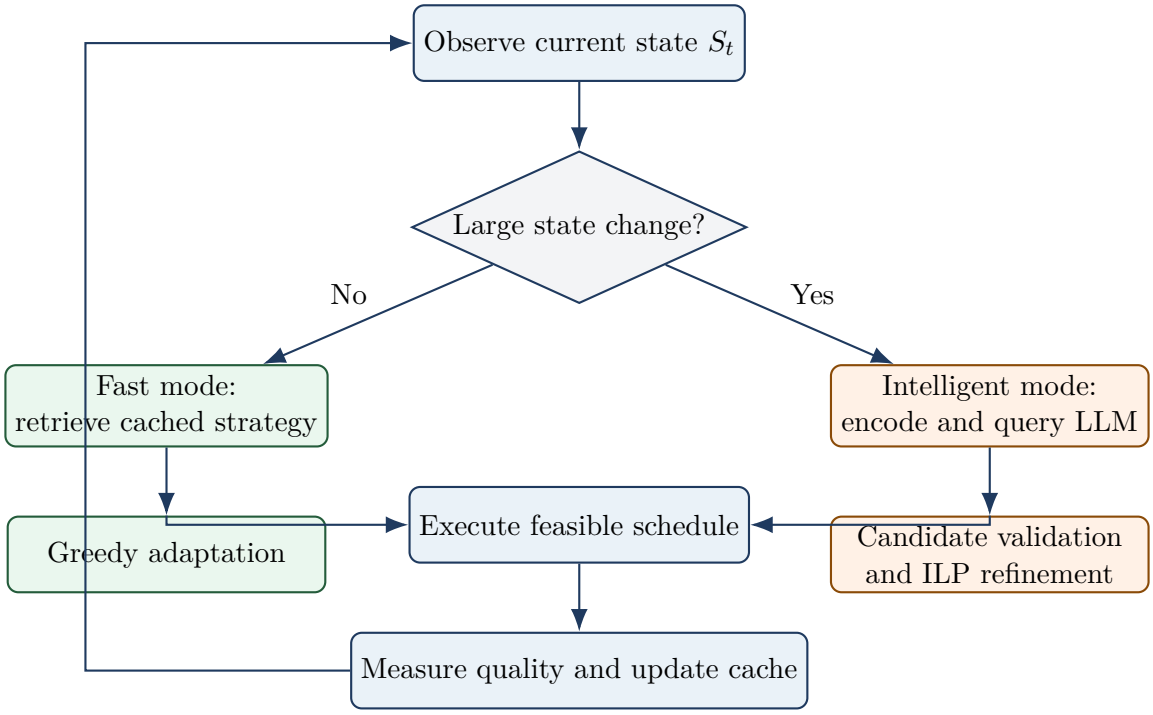
A direct LLM allocator would be responsible for exact numerical packing, all hard constraints, and the complete task-to-node mapping. LLMSched reduces that burden by asking the LLM to propose promising placements and then using a solver to make the result executable. However, the paper does not provide a controlled experiment proving that candidate-only generation is always faster than direct LLM allocation. Its strongest latency reduction comes from **not invoking the LLM for most decisions**.

2 Complete Closed-Loop Architecture

The architecture contains four operational modules:

1. **State encoder**: converts numerical, graph, workload, and SLA information into a structured textual representation.
2. **LLM reasoning engine**: produces multiple candidate schedules from the encoded state.
3. **ILP refinement module**: enforces exact constraints and improves the candidate schedule.
4. **Execution controller**: applies schedules, monitors outcomes, triggers rescheduling, and updates the strategy cache.

The system has two execution paths:



3 Step 1: Observe the Scheduling State

At scheduling time t, the scheduler observes a set of heterogeneous compute nodes

$$\mathcal{N} = \{n_1, n_2, \ldots, n_N\}.$$

Each node n_i has a capacity vector

$$\mathbf{c}_i = \left[c_i^{\text{CPU}}, c_i^{\text{MEM}}, c_i^{\text{DISK}}, c_i^{\text{NET}}\right]$$

and a current utilization vector

$$\mathbf{u}_i(t) \in [0,1]^4.$$

Each job is represented as a directed acyclic graph (DAG). The observable job state can include:

- arrival time, priority, deadline, and business weight;
- task-level CPU, memory, disk, network, or accelerator demands;
- estimated execution times on alternative node types;
- DAG precedence constraints;
- data-transfer sizes and locality preferences;
- affinity, anti-affinity, reservation, and availability-zone constraints.

The ideal scheduling output is a mapping

$$\sigma_t : \mathcal{T} \to \mathcal{N} \cup \{\perp\},$$

where $\perp$ means that a task remains pending.

4 Step 2: Decide Whether the LLM Is Needed

Calling an LLM for every scheduling event would dominate runtime and cost. LLMSched therefore computes the magnitude of state change:

$$\Delta(S_t, S_{t-1}) = \alpha \frac{\|U_t - U_{t-1}\|_2}{\|U_{t-1}\|_2} + \beta \frac{|J_t \triangle J_{t-1}|}{|J_{t-1}|} + \gamma I_{\text{SLA-risk}}. \tag{1}$$

Here:

- U_t is the cluster utilization vector;
- $J_t \triangle J_{t-1}$ is the symmetric difference between current and previous active-job sets;
- $I_{\text{SLA-risk}}$ indicates impending deadline or SLA risk.

The trigger is

$$\text{TriggerLLM} = \begin{cases} \text{True}, & \Delta(S_t, S_{t-1}) > \theta, \\ \text{False}, & \text{otherwise}, \end{cases} \tag{2}$$

with $\theta = 0.15$ in the paper.

Typical reasons to enter intelligent mode include:

- workload spikes or sharp utilization shifts;
- appearance of unusual job structures;
- approaching SLA violations;
- node failure or resource degradation;
- sudden deterioration in scheduling quality.

5 Step 3A: Fast-Mode Scheduling

Fast mode is used when the current state resembles a previously observed state. The strategy library is

$$\mathcal{L} = \{(S_i, \sigma_i, Q_i)\},$$

where S_i is an abstract state representation, σ_i is a validated schedule or policy, and Q_i is its measured quality.

The scheduler retrieves the best strategy according to

$$\sigma_{\text{cached}} = \arg\max_{\sigma_i \in \mathcal{L}} \text{Similarity}(S_t, S_i)\, Q_i. \tag{3}$$

The cached strategy is adapted because the current job and node sets are not identical to the stored state. A practical adaptation sequence is:

1. remove completed or cancelled jobs;
2. remove assignments to failed or unavailable nodes;
3. preserve still-feasible assignments where possible;
4. greedily place new or displaced tasks;
5. check capacity, locality, affinity, and priority constraints;
6. execute the adapted schedule.

The paper reports the following fast-mode median values:

Table 1: Reported fast-mode latency components.

Operation	Median latency
State encoding	1.2 ms
Cache lookup	0.3 ms
Greedy adjustment	2.7 ms
Fast-mode total	4.2 ms

Fast mode accounts for 96.8% of scheduling decisions in the reported evaluation.

6 Step 3B: Intelligent-Mode State Encoding

When the trigger fires, raw scheduling data are converted into a structured textual state. LLMSched uses four levels of representation.

6.1 Level 1: Cluster-Wide Summary

A compact summary communicates the operating regime:

```
CLUSTER STATE
Total nodes: 1000
Active nodes: 987
CPU utilization: 67.3%
Memory utilization: 72.1%
Pending jobs: 234
Running jobs: 1876
High-priority jobs: 45
Recent SLA violations: 12
```

6.2 Level 2: Resource Availability Clusters

Rather than enumerate every node, similar nodes are clustered and represented by profile summaries:

```
C1 [High-CPU]
200 nodes
Average available CPU: 12.4 cores
Average available memory: 45.2 GB
Zone: us-west-1a

C2 [Memory-Intensive]
200 nodes
Average available CPU: 8.1 cores
Average available memory: 98.7 GB
Zone: us-west-1b
```

This reduces context length while retaining coarse placement choices.

6.3 Level 3: Job DAG Representation

Dependencies and data flow are encoded textually:

```
JOB J1234
Priority: HIGH
Deadline: 180 s

T1: CPU=4, MEM=8 GB, EST=30 s -> {T3,T4}
T2: CPU=4, MEM=8 GB, EST=32 s -> {T3,T4}
T3: CPU=8, MEM=16 GB, EST=45 s -> {T5}
T4: CPU=8, MEM=16 GB, EST=43 s -> {T5}
T5: CPU=2, MEM=4 GB, EST=15 s -> END
```

6.4 Level 4: Explicit Constraints

Hard restrictions are translated into concise statements:

```
CONSTRAINTS
1. Job J1234 must run in zone us-west-1.
2. Replicated tasks T1 and T2 must use different nodes.
3. T3 requires at least 16 GB free memory.
4. Node N567 is reserved for User U89.
```

7 Step 4: Relevance Filtering and Context Reduction

The complete cluster state cannot be sent to the LLM. For each job J and node n_i, the paper defines a relevance score

$$\text{Relevance}(J, n_i) = \beta_1 \,\text{ResourceMatch}(J, n_i) + \beta_2 \,\text{LocalityScore}(J, n_i) + \beta_3 \,\text{HistoricalAffinity}(J, n_i). \tag{4}$$

Only the top-M nodes or node clusters are included. The purpose is to transform

$$\text{full cluster state} \rightarrow \text{small relevant search region} \rightarrow \text{shorter prompt and faster inference.}$$

The reported experiments limit prompts to approximately 8K tokens. This step is important for both latency and decision quality: sending more information is not always better if most of it is irrelevant or repetitive.

8 Step 5: Prompt Construction

For candidate i, the prompt is conceptually

$$P_i = P_{\text{system}} \oplus P_{\text{few-shot}} \oplus S_t \oplus P_{\text{reasoning}} \oplus P_{\text{constraints}}. \tag{5}$$

8.1 System Role

The model is instructed to act as an expert cloud scheduler balancing completion time, utilization, SLA compliance, locality, fairness, and resource constraints.

8.2 Few-Shot Examples

The prompt provides three to five examples illustrating good scheduling behavior, such as placing memory-heavy tasks on memory-rich nodes or co-locating tasks with data.

8.3 Structured Reasoning Questions

The model is asked to reason about:

1. urgency and deadline risk;
2. current resource bottlenecks;
3. opportunities to reduce data movement;
4. beneficial task co-location;
5. constraint satisfaction.

8.4 Constraint Reminders

Critical restrictions are repeated explicitly:

```
CRITICAL:
Task T3 requires at least 16 GB free memory.
Only nodes N45, N78, and N102 satisfy this condition.
```

The paper reports that constraint-aware prompting reduces invalid assignments substantially, but it does not guarantee feasibility. This is why the symbolic refinement stage is still necessary.

9 Step 6: Generate Multiple Candidate Schedules

The LLM is queried K times to obtain

$$\mathcal{C} = \{c_1, c_2, \ldots, c_K\}.$$

The reported configuration uses $K = 5$ with temperatures

$$\{0.3, 0.4, 0.5, 0.6, 0.8\}.$$

Lower temperatures favor conservative, repeatable decisions; higher temperatures encourage exploration. Each text response is parsed into task-node pairs:

```
Candidate 1
T1 -> N10
T2 -> N11
T3 -> N22
T4 -> N23
T5 -> N22
```

This stage produces **candidate assignments**, not final executable allocations.

10 Step 7: Candidate Validation

For each candidate c_i, the scheduler checks for:

- resource-capacity violations;
- illegal or incompatible nodes;
- broken affinity or anti-affinity rules;
- precedence violations;
- assignment of tasks to unavailable resources.

Let $V(c_i)$ be the set of violations. The appendix rejects a candidate if

$$|V(c_i)| > 0.3|\mathcal{T}|. \tag{6}$$

Candidates with fewer violations can be minimally corrected and passed to the solver as warm starts.

11 Step 8: Constraint Pruning

Before creating the ILP, impossible task-node pairs are removed. For task t, the feasible-node set is

$$F(t) = \left\{ n_j : d_t^r \le c_j^r (1 - u_j^r),\ \forall r \right\}. \tag{7}$$

This reduces the number of decision variables and avoids wasting solver time on obviously infeasible assignments. The paper reports an approximate 40% reduction in problem size from pruning.

12 Step 9: ILP Refinement

Define binary variables

$$x_{tj} = \begin{cases} 1, & \text{task } t \text{ is assigned to node } n_j, \\ 0, & \text{otherwise.} \end{cases}$$

The optimization objective combines execution quality and distance from the LLM proposal:

$$\min_x \sum_{t\in\mathcal{T}} \sum_{j=1}^{N} w_t e_t(n_j) x_{tj} + \lambda \sum_{t\in\mathcal{T}} \sum_{j=1}^{N} d_{\text{LLM}}(t, n_j) x_{tj}, \tag{8}$$

where

$$d_{\text{LLM}}(t, n_j) = \begin{cases} 0, & \text{if the LLM proposed } t \to n_j, \\ 1, & \text{otherwise.} \end{cases}$$

The reported value is $\lambda = 0.3$.

The LLM proposal is therefore a **soft preference**. The solver may change it when a different assignment is needed for feasibility or better objective value.

12.1 Core Constraints

One assignment per task

$$\sum_{j=1}^{N} x_{tj} = 1, \qquad \forall t \in \mathcal{T}. \tag{9}$$

Resource capacity

$$\sum_{t \in \mathcal{T}} d_t^r x_{tj} \leq c_j^r (1 - u_j^r), \qquad \forall j, r. \tag{10}$$

Affinity

$$x_{tj} = 0 \qquad \text{when } n_j \notin N_t. \tag{11}$$

Precedence and timing A child task may not start until its predecessors have completed. Additional constraints can represent reservations, anti-affinity, zones, hardware requirements, or fault tolerance.

13 Step 10: Accelerate the Solver

Four acceleration techniques are described:

1. **Warm start**: initialize the solver from a corrected LLM candidate. The paper reports 60–80% lower solve time.
2. **Time limit**: impose a strict 50 ms solver budget. If optimality is not proven, return the best feasible solution found.
3. **Decomposition**: split large DAGs into weakly connected subproblems and solve them independently or in parallel.
4. **Constraint pruning**: exclude impossible task-node pairs before optimization.

The observed median ILP refinement time is 18.4 ms.

14 Step 11: Select the Final Candidate

The paper is not fully explicit about how the K generated candidates are reduced to one final schedule. Algorithm 1 returns a candidate set, while Algorithm 2 describes refinement of a single candidate.

A clean implementation is:

$$\sigma_i^* = \text{ILPRefine}(c_i), \tag{12}$$

$$\sigma^* = \arg\min_{\sigma_i^*} \mathcal{L}(\sigma_i^*), \tag{13}$$

where $\mathcal{L}$ is the scheduler's multi-objective loss.

For large instances, a lower-cost alternative is to score all candidates approximately and refine only the best one or two. Any implementation should state this candidate-selection rule explicitly because it is underspecified in the paper.

15 Step 12: Fallback and Partial Scheduling

If the ILP cannot produce a feasible schedule, LLMSched may use:

- greedy bin packing;
- Kubernetes default scheduling;
- Dominant Resource Fairness (DRF);
- partial schedule execution, leaving some tasks pending;
- cached strategies for rapid failure recovery.

Fallback behavior is essential because an LLM call may time out, an output may be malformed, or the cluster may not have sufficient resources to place every task.

16 Step 13: Execute the Schedule

The execution controller dispatches the refined schedule, monitors progress, detects failures, and records outcomes. A final schedule may look like:

```
T1 -> N10
T2 -> N11
T3 -> N22
T4 -> N23
T5 -> pending until T3 and T4 complete
```

The scheduler should distinguish between a task being feasible in the final plan and a task being immediately runnable under DAG dependencies.

17 Step 14: Feedback and Strategy-Cache Update

After schedule σ_t is executed, a quality score is calculated:

$$Q(\sigma_t) = -\left[\alpha_1 \,\mathrm{JCT}_t + \alpha_2(1 - U_t) + \alpha_3 \,\mathrm{SVR}_t\right]. \tag{14}$$

The strategy library is then updated:

$$\mathcal{L} \leftarrow \mathcal{L} \cup \{(S_t, \sigma_t, Q(\sigma_t))\}. \tag{15}$$

Low-quality strategies are periodically removed. The resulting loop is

$$\text{reason} \rightarrow \text{optimize} \rightarrow \text{execute} \rightarrow \text{observe} \rightarrow \text{reuse or improve}.$$

18 Reported Latency Breakdown

The paper measures end-to-end wall-clock time on the scheduler side. The reported LLM latency includes request transmission, server-side inference, and response reception.

Table 2: Reported scheduling latency values.

Mode or component	P50	P95	P99	Frequency
Fast mode	4.2 ms	8.9 ms	13.5 ms	96.8%
LLM inference	342.7 ms	598.3 ms	623.8 ms	–
ILP refinement	18.4 ms	31.2 ms	43.7 ms	–
Intelligent mode total	362.3 ms	617.4 ms	645.1 ms	3.2%
Average over all decisions		15.7 ms		100%

The dominant reason for the low average is the execution mix:

$$0.968(4.2\text{ ms}) + 0.032(362.3\text{ ms}) \approx 15.7\text{ ms}. \tag{16}$$

Therefore, the core latency strategy is:

> Invoke the LLM only for significant state changes, reuse validated strategies for routine cases, keep the prompt compact, and delegate exact constraint handling to a fast bounded solver.

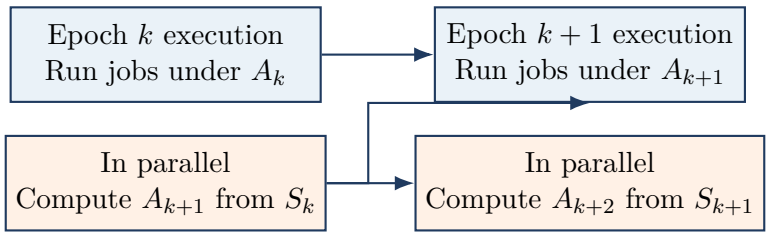
19 Full Intelligent-Mode Sequence

1. Observe jobs, nodes, utilization, deadlines, and constraints.
2. Encode the state hierarchically.
3. Filter to the most relevant jobs and nodes.
4. Insert explicit hard-constraint reminders.
5. Build the system, few-shot, state, and reasoning prompt.
6. Generate $K = 5$ candidate assignments.
7. Parse each response into a structured mapping.
8. Validate each candidate and identify violations.
9. Reject candidates with excessive violations.
10. Prune impossible task-node pairs.
11. Warm-start ILP from each remaining candidate.
12. Solve under a 50 ms limit.
13. Select the best feasible refined schedule.
14. Execute the schedule.
15. Observe JCT, utilization, SLA performance, and failures.
16. Store the state, strategy, and quality score for future reuse.

20 Adaptation to a Three-Minute Next-Epoch Simulator

In the proposed simulation, allocation A_k is active during epoch k while the scheduler computes allocation A_{k+1} for the next epoch. This is a pipelined control design:

$$A_{k+1} = \text{Scheduler}(S_k), \qquad T_{\text{decision}} \leq 180 \text{ s}. \tag{17}$$



The scheduler can use any amount of computation up to 180 seconds, but the allocation must be ready before the next boundary. The current epoch should continue executing during decision-making; otherwise, scheduling latency would directly waste cluster time.

20.1 Recommended Tick Semantics

At the start of epoch k:

1. freeze a snapshot S_k containing only currently observable information;
2. continue executing A_k for 180 seconds;
3. concurrently compute A_{k+1};

4. do not reveal future trace arrivals unless the experiment explicitly studies prediction;
5. activate A_{k+1} at the next tick boundary;
6. account for migration, resizing, checkpoint, or restart overhead.

20.2 Simulation Pseudocode

```
current_allocation = initial_allocation

for tick in range(number_of_ticks):
    state = observe_state_at_tick_start()

    # Conceptually concurrent with job execution.
    if significant_state_change(state):
        candidates = llm_generate_candidates(state, k=5)

        refined = []
        for candidate in candidates:
            if violation_ratio(candidate, state) <= 0.30:
                solution = ilp_refine(
                    candidate=candidate,
                    state=state,
                    time_limit_seconds=0.050,
                )
                if solution is not None:
                    refined.append(solution)

        if refined:
            next_allocation = min(refined, key=objective_value)
        else:
            next_allocation = fallback_scheduler(state)
    else:
        cached = retrieve_similar_strategy(state)
        next_allocation = greedy_adapt(cached, state)

    run_jobs(current_allocation, duration_seconds=180)
    current_allocation = next_allocation
```

In a real implementation, the scheduling branch and `run_jobs` execute concurrently. A discrete-event simulator may emulate this by computing the next allocation first while ensuring that its input snapshot is restricted to information available at the start of the current epoch.

21 Recommended Responsibility Split

The clearest division of labor is:

Table 3: Recommended responsibilities by component.

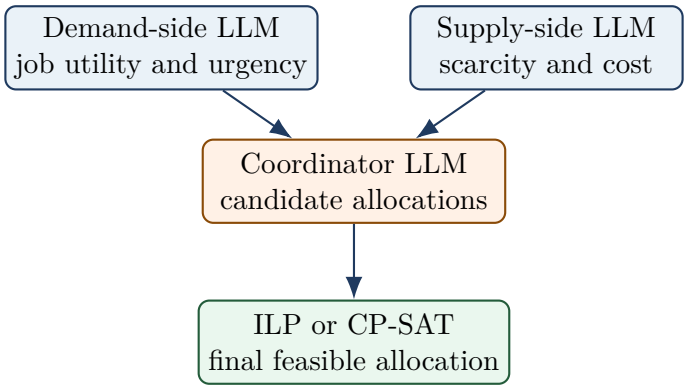
Component	Main responsibility
LLM	Identify promising allocation structures, urgent jobs, bottlenecks, locality opportunities, and candidate task-node mappings.
Formal optimizer	Calculate exact quantities and placements, guarantee hard constraints, account for capacity and reallocation costs, and return an executable schedule.
Execution controller	Apply the schedule, detect failures, record outcomes, and trigger the next decision.
Cache and fast heuristic	Reuse high-quality strategies when the state is routine and similar to previously observed states.

The compact summary is:

> The LLM proposes what looks promising; the optimizer decides what is exact and feasible.

22 Implications for a Demand-Supply Multi-Agent Extension

The same pipeline can be extended without making the LLM agents responsible for exact packing. A demand-side agent can estimate job urgency and marginal utility; a supply-side agent can estimate scarcity, fragmentation, and opportunity cost; a coordinator can combine those summaries into candidate allocations; and a formal solver can produce the final feasible assignment.



This preserves the main LLMSched principle: LLMs perform semantic valuation and strategic reasoning, while symbolic optimization guarantees correctness.

23 Key Caveats

- The paper does not fully specify whether all five candidates are refined or only one is selected before ILP.
- The reported latency numbers come from a simulated evaluation and synchronous GPT-4 API measurements; deployment latency may vary.
- Candidate generation is not proven to be universally faster than direct LLM allocation under equal token and call budgets.

- Fast average latency depends primarily on the 96.8% fast-mode rate, not on making the LLM itself millisecond-fast.
- In a trace simulator, future arrivals must remain hidden unless prediction or oracle access is explicitly part of the experiment.

24 Conclusion

LLMSched is best understood as a hybrid adaptive controller. It uses a language model only when cluster conditions change significantly, constrains the LLM input through hierarchical encoding and relevance filtering, requests several candidate schedules, repairs and optimizes those candidates with a time-bounded ILP solver, executes the result, and stores successful strategies for rapid reuse. In a three-minute next-epoch simulator, the same pipeline can run concurrently with the current allocation, provided that the next schedule is completed before the tick boundary and is based only on currently observable state.

References

[1] G. Ding, S. Yang, H. Lin, Z. Chen, and J. S. Yang, "LLM-Driven Adaptive Cloud Resource Scheduling: Bridging Reasoning Intelligence With Optimization Guarantees," *IEEE Open Journal of the Computer Society*, vol. 7, pp. 560–573, 2026. DOI: 10.1109/OJCS.2026.3667549.